Threshold detection and ROC curves

Laboratory 6, DEDP

Table of contents

1 Objective 1

2 Theoretical aspects 2

2.1 Binary decisions by thresholding . . . 2
2.2 Four possible outcomes . . . 2
2.3 Changing the threshold: the ROC curve . . . 3
2.4 How often is each bit transmitted? . . . 3
2.5 Matlab tools . . . 4
Generating random bits with chosen probabilities . . . 4
Transmitted and received voltages . . . 4
Comparing the received-voltage distributions . . . 5
Sweeping and selecting thresholds . . . 5
2.6 Summary: functions and notation you should know . . . 5

3 Exercises 7

4 Final questions 9

1 Objective

Simulate transmission of bits using two voltage levels with additive noise, make receiver decisions by thresholding, and measure detections and false alarms. Plot an ROC curve and choose a threshold subject to a false-alarm limit.

2 Theoretical aspects

2.1 Binary decisions by thresholding

A transmitter may represent each bit by a constant voltage level:

- bit 0: $s = 0$ V; (or some other value)
- bit 1: $s = A$ V, where $A > 0$.

The signal is corrupted by additive noise. The receiver receives the noisy signal, and takes one sample per transmitted bit, in order to make a decision. The received voltage is:

$$r = s + n,$$

where n is additive noise. The two hypotheses describe what was transmitted:

- H_0: bit 0 was transmitted ($s = 0$ V);
- H_1: bit 1 was transmitted ($s = A$ V).

The receiver does not know the transmitted bit. It compares r with a threshold T and makes one of two decisions:

$$D_1 \text{ (decide bit 1) if } r > T; \qquad D_0 \text{ (decide bit 0) if } r \leq T.$$

Hypotheses describe the true transmitted bit; decisions describe the receiver's output. Noise can cause the decision to be wrong.

In Matlab, `v = r > T` returns a logical vector: `true` represents D_1, and `false` represents D_0.

2.2 Four possible outcomes

Transmitted bit (hypothesis)	Decision D_0 (bit 0)	Decision D_1 (bit 1)
H_0 (bit 0)	Correct rejection	False alarm
H_1 (bit 1)	Miss	Hit (correct detection)

A false alarm means deciding bit 1 when bit 0 was transmitted; a miss means deciding bit 0 when bit 1 was transmitted. The conditional probabilities are $P_{\text{FA}} = P(D_1 \mid H_0)$ and $P_{\text{D}} = P(D_1 \mid H_1)$.

Let N_0 be the number of observations whose true state is H_0, and N_1 the number whose true state is H_1. Estimate the conditional probabilities by:

$$\hat{P}_{\mathrm{FA}} = \frac{\text{number of false alarms}}{N_0}, \qquad \hat{P}_{\mathrm{D}} = \frac{\text{number of hits}}{N_1}.$$

The estimated miss probability is $1-\hat{P}_{\mathrm{D}}$. **Do not divide both counts by the total N_0+N_1:** each probability refers to a different true state.

For example, keep observations from the two states in separate row vectors `r0` and `r1`. The number of false alarms is `sum(r0 > T)`, and its denominator is `numel(r0)`. Count hits in `r1` in the same way. We assume both vectors are nonempty.

2.3 Changing the threshold: the ROC curve

Lowering T makes decisions D_1 more frequent: both the detection probability and the false-alarm probability increase or stay unchanged. Raising T has the opposite effect.

The **receiver operating characteristic (ROC)** plots:

- horizontal axis: P_{FA};
- vertical axis: P_{D}.

Each threshold gives one point. Sweep through many thresholds to obtain the curve. Use the **same observations for every threshold**, so changes reflect the decision rule rather than newly generated noise.

For finite observations, `T = Inf` gives the point $(0, 0)$ and `T = -Inf` gives $(1, 1)$. These are the rules "always output D_0" and "always output D_1", respectively. An informative detector has points towards the upper-left corner: many detections with few false alarms. The diagonal $P_{\mathrm{D}} = P_{\mathrm{FA}}$ is a reference for decisions made randomly, independently of the true state.

A practical requirement might be $P_{\mathrm{FA}} \leq 0.05$. Among the tested thresholds satisfying this limit, select one with the largest estimated P_{D}. A limit met in a simulation is not a guarantee for future observations.

2.4 How often is each bit transmitted?

The ROC describes performance **within each true state**. The overall error probability also depends on how often each bit is transmitted.

If $p_1 = P(H_1)$ is the probability of transmitting bit 1, then $P(H_0) = 1 - p_1$, and:

$$P_{\mathrm{error}} = (1 - p_1)P_{\mathrm{FA}} + p_1(1 - P_{\mathrm{D}}).$$

Multiplying this probability by the number of future observations gives their expected number of errors. Changing p_1 alone does not change the ROC if the two conditional received-voltage distributions stay unchanged, but it can change which threshold gives fewer total errors. This formula treats a false alarm and a miss as equally costly.

2.5 Matlab tools

Generating random bits with chosen probabilities

Use `rand(1,N)` to generate `N` uniform samples between 0 and 1. Comparing each sample with 0.5 produces random bits with equal probabilities:

```
N = 20000;
bits = rand(1,N) > 0.5;
```

To change the proportions, adapt the comparison threshold. For example, to obtain approximately **30% zeros and 70% ones**:

```
bits = rand(1,N) > 0.3;
```

Samples at or below 0.3 become 0; samples above 0.3 become 1. Since `rand()` is uniform, the probabilities are $P(0) = 0.3$ and $P(1) = 0.7$. More generally, `rand(1,N) > p0` produces zeros with probability `p0` and ones with probability `1-p0`.

The probabilities are predetermined, but the actual counts fluctuate: this method does **not** guarantee exactly 30% zeros and 70% ones in a finite vector. Check the observed proportions with:

```
fraction0 = sum(bits == 0)/N;
fraction1 = sum(bits == 1)/N;
```

This bit-generation threshold controls the transmitted bit probabilities; it is different from the receiver's voltage threshold T.

Transmitted and received voltages

Multiply the bit vector by A to obtain the transmitted voltages. Use `randn()` as in Lab 3 to generate independent Gaussian noise and add it to the transmitted voltages to obtain the received vector `r`.

Logical indexing separates received samples according to the **transmitted** bits, not the receiver's decisions:

```
r0 = r(bits == 0);
r1 = r(bits == 1);
```

`==` tests equality element by element; `=` assigns a value to a variable.

Comparing the received-voltage distributions

To compare two histograms on the same axes:

```
edges = -4:0.25:7;
figure
histogram(r0, edges, 'Normalization', 'pdf')
hold on
histogram(r1, edges, 'Normalization', 'pdf')
hold off
xlabel('Received voltage (V)')
ylabel('Estimated probability density')
legend('Bit 0 transmitted', 'Bit 1 transmitted')
```

`edges` specifies common bin boundaries. The plotting range does not truncate the vectors: use all observations when calculating probabilities.

Sweeping and selecting thresholds

For a threshold sweep, create a vector such as `thresholds = -4:0.05:7`. Preallocate result vectors with `zeros(size(thresholds))`, then use a `for` loop to fill one entry per threshold. Plot the ROC with `plot(pFA,pD)`.

Logical indexing selects entries satisfying a condition. For example:

```
eligible = pFA <= 0.05;
eligibleThresholds = thresholds(eligible);
```

Apply the same logical index to `pD` to obtain the matching detection probabilities. `[bestValue,index] = max(v)` returns the largest value and its first index **within** `v`, not within the original unfiltered vector.

2.6 Summary: functions and notation you should know

Signature or notation	Example	Description
`rand(1,N) > p0`	`bits = rand(1,N) > 0.3`	Random bits: probability `p0` of 0 and `1-p0` of 1
`A*bits`	`s = 3*bits`	Map bits to transmitted voltage levels 0 and A
`r(bits == 0)`	`r0 = r(bits == 0)`	Select received samples whose transmitted bit was 0
`mu + sigma*randn(1,N)`	`3 + randn(1,10000)`	Normal samples with mean `mu` and standard deviation `sigma`
`r > T`	`decision = r > 1.5`	Logical decisions: 1 for D_1, 0 for D_0
`sum(condition)`	`sum(r0 > T)`	Count observations satisfying a condition
`numel(x)`	`numel(r0)`	Number of observations in a vector
`histogram(x,edges,...)`	`histogram(r0,edges,'Normalization','pdf')`	Histogram with specified bin boundaries and unit-area normalization
`hold on`, `hold off`, `legend(...)`	`legend('Bit 0 transmitted','Bit 1 transmitted')`	Overlay plots and identify them
`a:step:b`	`-4:0.05:7`	Candidate thresholds with a fixed step
`zeros(size(v))`	`pFA = zeros(size(thresholds))`	Preallocate a vector with the same dimensions as `v`
`for k = 1:numel(v)`	`T = thresholds(k)` inside the loop	Visit each candidate by index
`Inf`, `-Inf`	`thresholds = [-Inf thresholds Inf]`	Add thresholds giving the two ROC endpoints
`plot(x,y)`	`plot(pFA,pD)`	Plot detection probability against false-alarm probability
Logical indexing	`pD(pFA <= 0.05)`	Select matching entries using a condition
`max(v)`	`[bestValue,index] = max(v)`	Largest value and the index of its first occurrence

3 Exercises

Write your commands in a script or live script. Label figures and record your numerical results and short explanations in comments. Use the same generated vectors throughout Exercises 1–4; do not generate new received samples inside the threshold loop.

1. Simulate transmission using two voltage levels:

   Use $A = 3$ V. The channel adds independent Gaussian noise with mean 0 V and standard deviation 1 V to each transmitted voltage. Thus, received samples follow $\mathcal{N}(0, 1)$ under H_0 and $\mathcal{N}(3, 1)$ under H_1, with voltages expressed in volts. Each sample leads to one bit decision.

   a. Generate a row vector `bits` of 20000 random bits. Call `rand()` and compare its output with 0.5 so that each bit has a 50% chance of being 0 or 1. Count the 0s and 1s. Are the counts exactly equal?

   b. Map `bits` to a vector `s` of transmitted voltages, using 0 V for bit 0 and 3 V for bit 1. Generate a noise vector of the same size using `randn()`, then form the received vector `r = s + n`.

   c. Extract `r0` and `r1` from `r` using the transmitted bits as logical conditions. Plot their PDF-normalized histograms on the same axes, using common bin boundaries. Describe the overlap. Can a threshold separate all received samples correctly?

   d. Before calculating, predict what happens to false alarms and misses when the threshold increases.

2. Evaluate individual thresholds:

   a. Set `T = 1.5`. Count false alarms and correct rejections in `r0`, then hits and misses in `r1`. Verify that the two counts for each true state add up to the number of samples for that transmitted bit.

   b. Estimate `pFA`, `pD`, and the miss probability, using the appropriate denominator for each. Also calculate the total number of errors.

   c. Repeat for `T = 1` and `T = 2`. Record a small table with one row per threshold and columns for `T`, `pFA`, `pD`, and the total error count. Compare with your prediction from Exercise 1.

3. Draw the ROC and choose an operating point:

   a. Create candidate thresholds from -4 to 7 in steps of 0.05. Add `-Inf` at the beginning and `Inf` at the end to include the ROC endpoints.

b. Complete the template below to calculate one pair of probabilities per threshold. Replace each `...` with the appropriate expression, using the counting method from Exercise 2. Do not generate new noise inside the loop.

```
pFA = zeros(size(thresholds));
pD = zeros(size(thresholds));

for k = 1:numel(thresholds)
    T = thresholds(k);

    % Count false alarms and hits for this threshold
    falseAlarms = ...;
    hits = ...;

    % Divide by the number of samples for the corresponding true bit
    pFA(k) = ...;
    pD(k) = ...;
end
```

c. Plot `pD` against `pFA`. Add the diagonal reference with `plot([0 1],[0 1],'--')`, using `hold on`. Label both axes. Verify the endpoints and explain the direction in which the curve is traversed as the threshold increases.

d. Among the tested thresholds with `pFA <= 0.05`, find one with the largest `pD`. Report the threshold and both probabilities, and mark its point on the ROC using `plot(chosenFA,chosenD,'ro')`. If several thresholds give the same largest detection probability, choose the first in the threshold vector.

4. **Optional: unequal bit probabilities**:

The bits above were generated with equal probabilities. Now consider future transmissions in which the probability of bit 1 may differ, while the voltage levels and noise distribution stay unchanged.

a. Using the probabilities estimated in Exercise 2 for `T = 1`, `1.5`, and `2`, calculate the expected number of errors per 1000 future transmitted bits when $p_1 = 0.5$. Which of these three thresholds gives fewer expected errors?

b. Repeat for $p_1 = 0.01$, without generating new received samples. Calculate expected false alarms and expected misses separately, then add them. Which of the three thresholds is now preferred? Expected counts need not be integers.

c. Does the ROC change when only the probability of transmitting bit 1 changes? Explain why a smaller total error count can coexist with a larger fraction of missed 1 bits.

4 Final questions

1. If every received voltage increases by 10 V, how should the threshold change to keep every decision unchanged? Would the ROC change?
2. Can a detector have no false alarms in 10000 simulated samples for transmitted bit 0 even though its true false-alarm probability is positive?
3. A receiver always outputs D_0 (bit 0). What are its detection and false-alarm probabilities? Why could its overall error rate still look small when bit 1 is rarely transmitted?
4. What does an ROC curve fail to tell you about the consequences of a missed 1 bit compared with a false alarm?